

1

Understanding Spark

Apache Spark is a powerful open source processing engine originally developed by Matei Zaharia as a part of his PhD thesis while at UC Berkeley. The first version of Spark was released in 2012. Since then, in 2013, Zaharia co-founded and has become the CTO at Databricks; he also holds a professor position at Stanford, coming from MIT. At the same time, the Spark codebase was donated to the Apache Software Foundation and has become its flagship project.

Apache Spark is fast, easy to use framework, that allows you to solve a wide variety of complex data problems whether semi-structured, structured, streaming, and/or machine learning / data sciences. It also has become one of the largest open source communities in big data with more than 1,000 contributors from 250+ organizations and with 300,000+ Spark Meetup community members in more than 570+ locations worldwide.

In this chapter, we will provide a primer to understanding Apache Spark. We will explain the concepts behind Spark Jobs and APIs, introduce the Spark 2.0 architecture, and explore the features of Spark 2.0.

The topics covered are:

- What is Apache Spark?
- Spark Jobs and APIs
- Review of Resilient Distributed Datasets (RDDs), DataFrames, and Datasets
- Review of Catalyst Optimizer and Project Tungsten
- Review of the Spark 2.0 architecture

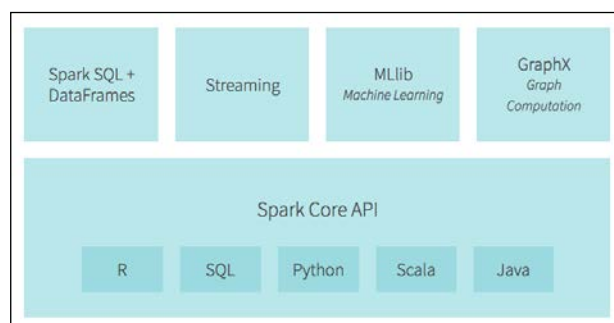
What is Apache Spark?

Apache Spark is an open-source powerful distributed querying and processing engine. It provides flexibility and extensibility of MapReduce but at significantly higher speeds: Up to 100 times faster than Apache Hadoop when data is stored in memory and up to 10 times when accessing disk.

Apache Spark allows the user to read, transform, and aggregate data, as well as train and deploy sophisticated statistical models with ease. The Spark APIs are accessible in Java, Scala, Python, R and SQL. Apache Spark can be used to build applications or package them up as libraries to be deployed on a cluster or perform *quick* analytics interactively through notebooks (like, for instance, Jupyter, Spark-Notebook, Databricks notebooks, and Apache Zeppelin).

Apache Spark exposes a host of libraries familiar to data analysts, data scientists or researchers who have worked with Python's `pandas` or R's `data.frames` or `data.tables`. It is important to note that while Spark DataFrames will be *familiar* to `pandas` or `data.frames` / `data.tables` users, there are some differences so please temper your expectations. Users with more of a SQL background can use the language to shape their data as well. Also, delivered with Apache Spark are several already implemented and tuned algorithms, statistical models, and frameworks: MLlib and ML for machine learning, GraphX and GraphFrames for graph processing, and Spark Streaming (DStreams and Structured). Spark allows the user to combine these libraries seamlessly in the same application.

Apache Spark can easily run locally on a laptop, yet can also easily be deployed in standalone mode, over YARN, or Apache Mesos - either on your local cluster or in the cloud. It can read and write from a diverse data sources including (but not limited to) HDFS, Apache Cassandra, Apache HBase, and S3:



Source: Apache Spark is the smartphone of Big Data <http://bit.ly/1QsgaNj>



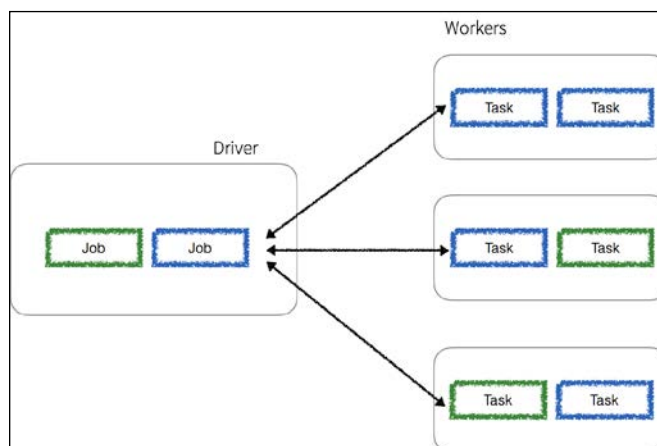
For more information, please refer to: Apache Spark is the Smartphone of Big Data at <http://bit.ly/1QsgaNj>

Spark Jobs and APIs

In this section, we will provide a short overview of the Apache Spark Jobs and APIs. This provides the necessary foundation for the subsequent section on Spark 2.0 architecture.

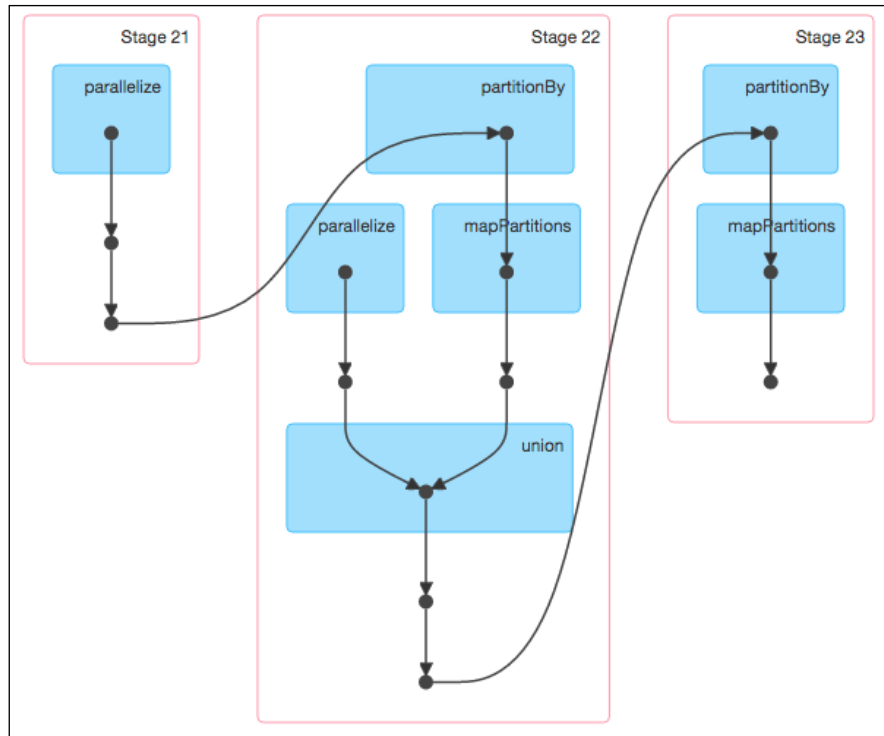
Execution process

Any Spark application spins off a single driver process (that can contain multiple jobs) on the *master* node that then directs executor processes (that contain multiple tasks) distributed to a number of *worker* nodes as noted in the following diagram:



The driver process determines the number and the composition of the task processes directed to the executor nodes based on the graph generated for the given job. Note, that any worker node can execute tasks from a number of different jobs.

A Spark job is associated with a chain of object dependencies organized in a direct acyclic graph (DAG) such as the following example generated from the Spark UI. Given this, Spark can optimize the scheduling (for example, determine the number of tasks and workers required) and execution of these tasks:



[ For more information on the DAG scheduler, please refer to <http://bit.ly/29WTiK8>.]

Resilient Distributed Dataset

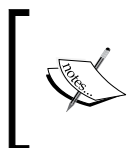
Apache Spark is built around a distributed collection of immutable Java Virtual Machine (JVM) objects called Resilient Distributed Datasets (RDDs for short). As we are working with Python, it is important to note that the Python data is stored within these JVM objects. More of this will be discussed in the subsequent chapters on RDDs and DataFrames. These objects allow any job to perform calculations very quickly. RDDs are calculated against, cached, and stored in-memory; a scheme that results in orders of magnitude faster computations compared to other traditional distributed frameworks like Apache Hadoop.

At the same time, RDDs expose some coarse-grained transformations (such as `map(...)`, `reduce(...)`, and `filter(...)` which we will cover in greater detail in Chapter 2, *Resilient Distributed Datasets*), keeping the flexibility and extensibility of the Hadoop platform to perform a wide variety of calculations. RDDs apply and log transformations to the data in parallel, resulting in both increased speed and fault-tolerance. By registering the transformations, RDDs provide data lineage - a form of an ancestry tree for each intermediate step in the form of a graph. This, in effect, guards the RDDs against data loss - if a partition of an RDD is lost it still has enough information to recreate that partition instead of simply depending on replication.



If you want to learn more about data lineage check this link
<http://ibm.co/2ao9B1t>.

RDDs have two sets of parallel operations: *transformations* (which return pointers to new RDDs) and *actions* (which return values to the driver after running a computation); we will cover these in greater detail in later chapters.



For the latest list of transformations and actions, please refer to the Spark Programming Guide at <http://spark.apache.org/docs/latest/programming-guide.html#rdd-operations>.

RDD transformation operations are *lazy* in a sense that they do not compute their results immediately. The transformations are only computed when an action is executed and the results need to be returned to the driver. This delayed execution results in more fine-tuned queries: Queries that are optimized for performance. This optimization starts with Apache Spark's DAGScheduler - the stage oriented scheduler that transforms using *stages* as seen in the preceding screenshot. By having separate RDD *transformations* and *actions*, the DAGScheduler can perform optimizations in the query including being able to avoid *shuffling* the data (the most resource intensive task).

For more information on the DAGScheduler and optimizations (specifically around narrow or wide dependencies), a great reference is the *Narrow vs. Wide Transformations* section in *High Performance Spark* in Chapter 5, *Effective Transformations* (<https://smile.amazon.com/High-Performance-Spark-Practices-Optimizing/dp/1491943203>).